

Informe N°2, MCDI500 PROGRAMACIÓN PARA LA CIENCIA DE DATOS

Avance Fase 3 - Semana 2: Núcleo algorítmico, eficiencia e implementación orientada a objetos

Proyecto: uso de redes sociales y salud mental percibida en adolescentes (YRBS 2023)

Docente: Dr. Omar Salinas Silva

Estudiantes: Matías Manriquez Ortiz, Daniel Pérez Ramirez, Abigail Robles Chávez, Roberto Sánchez Saldivia.

Repositorio GitHub: <https://github.com/MatiasManriquezO/proyecto-grupo8-mcdi500>

Magíster en Ciencia de Datos e Inteligencia Artificial

Universidad Andrés Bello, Santiago.

Santiago, 28 de septiembre de 2026.

Índice

Índice.....	2
Introducción.....	3
II. Diseño de soluciones algorítmicas eficientes.....	5
a. Codificación funcional y arquitectura básica del script.....	5
b. Preprocesamiento y transformación del dataset.....	7
c. Validación técnica y verificación del código.....	9
d. Eficiencia y optimización.....	12
e. Diseño estructurado del código.....	18
III. Implementación de código modular y robusto.....	20
a. Programación orientada a objetos.....	20
b. Documentación de arquitectura y funcionalidad del código.....	23
IV. Repositorio GitHub (F3).....	26
V. Notebooks ejecutables (F3).....	28
VI. Referencias.....	30
Anexos.....	32

Introducción

Este informe documenta el avance de la Fase 3 del proyecto transversal del Grupo 8, cuya pregunta de investigación es qué relación existe entre los patrones de uso de tecnología y los indicadores de salud mental autopercebida y horas de sueño en los estudiantes del Youth Risk Behavior Survey 2023 (Centers for Disease Control and Prevention [CDC], 2024). En la Fase 2 se construyó un pipeline funcional de obtención, limpieza y transformación; en el avance formativo de esta semana se escribieron scripts estructurados y recursivos y se midió su costo en tiempo y memoria. Esta entrega integra ambos resultados y les da forma de sistema: no hay datos nuevos ni un pipeline nuevo; lo que cambia es la arquitectura del código.

El entregable central es el notebook `F3/notebooks/S2_F3_NucleoAlgoritmico_P00_Grupo8.ipynb`, ejecutado completo con numeración continua de 1 a 54 y sin errores. El pipeline de la Fase 2 se reorganizó en una jerarquía de clases que vive en `F3/src/` y que el notebook importa, sin redefinirla: cada paso es una clase hija de `Transformador`, los pasos se encadenan con `Pipeline` y el tratamiento de valores faltantes se resuelve con el patrón `Strategy`. La prueba de que la reorganización no alteró nada es objetivo: el pipeline de clases, ejecutado sobre el archivo original del CDC, reproduce el conjunto procesado de la Fase 2 idéntico carácter por carácter.

La entrega incorpora además la retroalimentación de la Formativa 3 (67 de 69 puntos). Se dejó **un solo notebook central** y el de la Formativa pasó a una subcarpeta histórica; se declaró qué contiene cada una de las dos carpetas `src`; se trasladaron todas las clases a los módulos que proyectó la Tabla 8 de la Formativa 3, de modo que el notebook ya no define ninguna; y el ejemplo de Fibonacci, útil como ilustración, pero ajeno al YRBS, se reemplazó por un cálculo propio del proyecto que se beneficia de memoización: la prevalencia de salud mental no buena por segmentos anidados de uso de redes, sexo y edad (apartados II.d y III.b).

El documento sigue la estructura exigida: el apartado II presenta el diseño de soluciones

algorítmicas (codificación, preprocesamiento, validación, eficiencia y diseño estructurado); el III, la implementación orientada a objetos y la documentación de la arquitectura; el IV, el repositorio; el V, el notebook; y el VI, la bibliografía. El informe no repite el código: explica qué se construyó, cómo se verificó, qué se midió y por qué se tomaron las decisiones. Las cifras corresponden a la ejecución versionada del notebook y a los archivos guardados en F3/resultados/.

II. Diseño de soluciones algorítmicas eficientes

a. Codificación funcional y arquitectura básica del script

El código se organizó en tres niveles con responsabilidades separadas. En el primero están las funciones de entrada de `F3/src/carga.py`, que no conocen el pipeline: `leer_archivo` elige el lector según la extensión, `verificar_esquema` comprueba que existan las columnas declaradas en la celda de configuración y `cargar` combina ambas, avisando con un mensaje explícito si el archivo no existe. En el segundo están las **clases de transformación y análisis** de `F3/src/`, con interfaces fijas: `ajustar(df)` / `transformar(df)` para los pasos del pipeline y `explorar()` / `tabla()` para el núcleo analítico. En el tercero, la **orquestación**: Pipeline recorre los pasos en orden y la función fábrica `construir_pipeline_proyecto` (`F3/src/fabrica.py`) los arma a partir de las constantes de `src/procesamiento.py`, el mismo módulo que usó la Fase 2.

Todas las rutas y nombres de columna se declaran una sola vez en la celda de configuración (`RAIZ`, `RUTA_DATOS`, `RUTA_ORIGINAL`, `JERARQUIA_SEGMENTOS`, `SEMILLA = 42`). La raíz del repositorio se localiza con `encontrar_raiz()`, por lo que el notebook funciona desde cualquier carpeta. El flujo principal del proyecto queda reducido a pocas líneas:

```
from fabrica import construir_pipeline_proyecto      # F3/src/fabrica.py
from observadores import Bitacora                  # F3/src/observadores.py
from segmentacion import ArbolPrevalencia           # F3/src/segmentacion.py

# Fábrica: arma los 12 pasos de la Fase 2 como objetos Transformador
pipeline = construir_pipeline_proyecto(observable=True).suscribir(Bitacora())
proyecto = pipeline.ajustar(original).transformar(original) # 20.103 x 117 -> 20.103 x 26

# Núcleo analítico: prevalencia por segmentos (recursión + memoización)
arbol = ArbolPrevalencia(datos, JERARQUIA_SEGMENTOS, "salud_mental_mala", n_minimo=30)
tabla = arbol.tabla()                                # 121 segmentos
```

Tabla 1. Componentes del flujo principal

Componente	Recibe	Devuelve	Responsabilidad
cargar(ruta, columnas_esperadas)	ruta al CSV de F2	DataFrame 20.103 × 26	Leer el archivo y verificar el esquema declarado
perfilar(df)	DataFrame	DataFrame: tipo, únicos, % nulos	Perfil genérico, reutilizable en cualquier conjunto
Transformador.ajustar(df)	DataFrame	self (encadenable)	Validar columnas y aprender parámetros
Transformador.transformar(df)	DataFrame	copia transformada	Aplicar lo aprendido sin modificar la entrada
construir_pipeline_proyecto(...)	estrategia, observable	Pipeline de 12 pasos	Armar el pipeline desde las constantes de F2
ArbolPrevalencia.tabla()	DataFrame, jerarquía, n mínimo	DataFrame de segmentos	Prevalencia por segmentos anidados
comparar(implementaciones, ...)	dict de funciones	DataFrame de tiempos	Verificar igualdad y luego medir tiempo y memoria

Nota. Elaboración propia a partir de las secciones 0, 11, 13 y 14 del notebook y de los módulos de F3/src/.

Las convenciones son uniformes: nombres descriptivos en español, *docstring* en cada módulo, clase y función, comentarios solo donde la lógica no es evidente (por ejemplo, la copia defensiva de parametros o el uso de `type(self).__name__`) y validación de la entrada **antes** de trabajar. Una columna inexistente, un código fuera del *codebook* o un orden de llamadas incorrecto producen una excepción con el nombre del paso que falló, no un resultado silenciosamente incorrecto. Se evitó el patrón de funciones con banderas booleanas que ejecutan ramas distintas, señalado en el material como violación de la responsabilidad única.

El flujo es verificable porque cada paso deja registro de sí mismo. Con el patrón Observer, el pipeline notifica filas, columnas y tiempo al terminar cada paso: el archivo original pasa de 117 a 116 columnas al eliminar `orig_rec`, a 11 al seleccionar, a 13 con la bandera de faltantes, a 21 con la codificación de raza/etnicidad y a 26 con los cinco indicadores, conservando siempre las 20.103 filas. La ejecución completa de los 12 pasos tardó alrededor de 0,07 s (Anexo B).

b. Preprocesamiento y transformación del dataset

Siguiendo la guía de la Sumativa 2, el preprocesamiento de la Fase 2 no se rehízo: se encapsuló. Cada paso del notebook de F2 pasó a ser una clase hija de Transformador, definida en `F3/src/transformadores.py`, que conserva la decisión registrada en la bitácora. El pipeline parte del archivo original del CDC (20.103×117) y sus pasos, en orden, son los de la Tabla 2.

Tabla 2. Pasos del pipeline de la Fase 2 encapsulados en clases

#	Clase (paso de F2)	Transformación y justificación técnica	Evidencia de la ejecución
1	EliminadorColumnaVacía	Elimina <code>orig_rec</code> solo si está 100 % vacía; si tuviera un dato, se niega y avisa (bitácora 2.1)	100,0 % nulos; 117 → 116 columnas
2	SeleccionadorColumnas	Selecciona y renombra las 11 columnas de análisis declaradas en <code>proc.COLUMNAS_ANALISIS</code> ; exige record único (2.3)	116 → 11 columnas
3	MarcadorFaltantes	Crea <code>n_faltantes_analisis</code> y <code>caso_completo</code> antes de cambiar tipos, sobre los valores originales (2.5)	10.941 casos completos en las 7 variables
4	TratamientoFaltantes + ConservarFaltantes	Los faltantes se conservan como NA: son <i>no responde</i> genuino y no aleatorio; imputar inventaría respuestas (2.5)	13.813 faltantes antes y después
5	CastearCodigos	Códigos <code>float64</code> → <code>Int64</code> (entero con NA); detiene el proceso si aparece un código no entero (2.4)	7 columnas convertidas
6	ConversorOrdinal	Declara el orden del <i>codebook</i> con <code>pd.Categorical(ordered=True)</code> ; evita aritmética inválida sobre escalas (2.4, 2.7)	5 escalas ordinales
7	CodificadorOneHot	<code>raceeth</code> → 8 columnas 0/1 con nombres declarados; <NA> donde falta, nunca 0 (2.7)	13 → 21 columnas
8–12	IndicadorBinario (×5)	Indicadores 0/1 de sexo, salud mental no buena ($q84 = 4-5$), uso frecuente de redes, sueño ≥ 8 h y actividad ≥ 5 días	21 → 26 columnas; % positivos entre 23,0 y 78,1

Nota. Elaboración propia a partir de la sección 11 del notebook y de `F3/resultados/parametros_pipeline_f3.csv`.

Manejo de valores NA. El perfil inicial muestra faltantes en todas las variables de análisis, desde 0,49 % en `edad_cod` hasta 24,37 % en `redes_sociales_cod`; la variable objetivo `salud_mental_cod` tiene

4.398 faltantes (21,9 %). La decisión de la Fase 2 se mantiene y ahora queda dentro del código: el paso `TratamientoFaltantes` recibe la estrategia `ConservarFaltantes`. La comparación con las alternativas evaluadas en F2 se repitió cambiando solo el argumento de estrategia (Tabla 3): rellenar con la moda o la mediana inventa 4.398 respuestas de salud mental y desplaza la distribución observada hasta 15,3 puntos porcentuales, por lo que se descartan.

Tabla 3. Efecto de cada estrategia de faltantes sobre la variable objetivo

Estrategia	Respuestas usadas	Respuestas inventadas	Desvío máximo (pp)
conservar NA (proyecto)	15.705	0	0,0
moda	20.103	4.398	15,3
mediana	20.103	4.398	15,3

Nota. Desvío máximo respecto de la distribución observada de `salud_mental_cod`. Elaboración propia a partir de la sección 16 del notebook.

Casting, normalización y escalamiento. Los códigos se convierten a `Int64` y las cinco ordinales a categóricas ordenadas. No se normalizaron ni escalaron las variables del proyecto: sus códigos son categorías ordinales o nominales, no magnitudes, y un escalamiento estándar supondría distancias iguales entre categorías que la escala no garantiza (bitácora 2.4). El escalamiento (`EscaladorEstandar`) y la imputación (`ImputadorMediana`) se implementaron como clases didácticas, separadas de los pasos del proyecto en `F3/src/transformadores.py` (`CLASES_DIDACTICAS`), y se probaron en las secciones 2 a 10 del notebook, junto con la separación entrenamiento/prueba (12.564 / 3.141 registros con la variable objetivo-respondida) para demostrar que los parámetros se aprenden solo del entrenamiento: la media de `edad_cod` escalada en prueba es $-0,013$ y no cero. Tampoco se ponderaron con `weight`, `stratum` y `psu`, que se conservan para la Fase 4 (bitácora 2.6).

Codificación de las variables nominales. `raceeth_cod` es nominal: sus ocho códigos identifican grupos sin orden entre sí, por lo que tratarlos como número haría que un código 6 «valiera más» que un 2. Por eso `CodificadorOneHot` la convierte en ocho columnas 0/1 con nombres declarados (`raza_blanca`, `raza_hispana`, etc.). El vocabulario se toma del *codebook* y no de los datos: un código no declarado detiene el proceso, y un estudiante sin dato queda con `<NA>` en las ocho columnas, nunca con 0, que

afirmaría que no pertenece a ningún grupo. `sexo_cod` también es nominal, pero binaria; en lugar de dos columnas complementarias, que serían redundantes porque una se deduce de la otra, se crea un solo indicador (`sexo_femenino`) con `IndicadorBinario`. En ambos casos se conserva la columna original para trazabilidad.

Separación entrenamiento/prueba. De los 15.705 estudiantes que respondieron la variable objetivo se separó un 80 % para entrenamiento y un 20 % para prueba (12.564 / 3.141 registros, semilla 42). Esta separación se aplicó a las clases didácticas que sí aprenden valores de los datos, como la mediana o la media, para demostrar que se aprenden solo del entrenamiento. El pipeline del proyecto no la necesita en esta fase: es exploratorio y ninguno de sus pasos aprende un valor que luego modifique otros datos, pues solo validan contra valores declarados del *codebook*. Si en la Fase 4 se ajusta un modelo, la separación se incorporará antes del ajuste.

Impacto en el dataset final. El pipeline transforma el archivo del CDC de 20.103×117 a 20.103×26 : las 11 columnas de análisis, 2 banderas de faltantes, 8 columnas de raza/etnicidad y 5 indicadores. No se elimina ninguna fila ni se inventa ningún valor: se conservan los 13.813 faltantes de las 7 variables de análisis, 10.941 estudiantes (54,4 %) tienen las siete respuestas y 15.705 (78,1 %) respondieron la variable objetivo. Cada análisis posterior decide con qué subconjunto trabajar usando la bandera `caso_completo`, en lugar de heredar una eliminación de filas hecha antes de tiempo.

Validación del dataset resultante. Sobre la salida del pipeline se comprobó que se conservan las 20.103 filas; que los faltantes de las 7 variables son idénticos antes y después (13.813); que identificador y diseño muestral quedan intactos; que las columnas de raza/etnicidad suman exactamente 1 en las 19.733 filas con dato y son <NA> en los 370 restantes; y que la bandera `caso_completo` es coherente con el conteo de faltantes. El resultado final se compara además con el archivo oficial de la Fase 2 (apartado II.c).

c. Validación técnica y verificación del código

La verificación se organizó en los tres escenarios del material de la Semana 2 —caso normal,

casos límite y excepciones— mediante sentencias `assert` que detienen la ejecución en el punto donde se origina el error. Se aplicó a las clases didácticas (sección 7), al pipeline real del proyecto y al núcleo analítico (sección 15), que son los que resume la Tabla 4.

Tabla 4. Evidencia de verificación del pipeline y del núcleo analítico

Escenario	Qué se comprobó	Resultado
Normal	Filas conservadas; faltantes idénticos; id y diseño muestral intactos; raza suma 1; bandera coherente	5 comprobaciones superadas
Normal	Indicador de la clase <code>IndicadorBinario</code> frente a la columna <code>salud_mental_mala</code> de F2	Idénticos (4.398 NA conservados)
Normal	Nivel 1 del árbol de segmentos frente a la tabla de contingencia calculada con <code>bincount</code>	Mismos 8 porcentajes por dos caminos independientes
Límite	Conjunto de una sola fila · estudiante sin ninguna de las 7 respuestas · una sola categoría de raza	1 × 26 · 7 faltantes e indicadores <NA> (no 0) · 8 columnas creadas
Límite	Estrategia intercambiada por <code>RellenarModa</code>	Sin faltantes; ningún otro paso cambia
Límite	<code>n_minimo</code> mayor que el total · faltantes en una variable de la jerarquía	Solo la raíz · los segmentos de nivel 1 suman 11.602, sin segmento «faltante»
Excepción	12 entradas inválidas en el pipeline y el núcleo (Anexo C)	Cada una lanza la excepción esperada, con mensaje que nombra el componente
Contra F2	<code>pd.testing.assert_frame_equal</code> y comparación del texto CSV completo	Tablas iguales (20.103 × 26) e idénticas carácter por carácter

Nota. Elaboración propia a partir de las secciones 11, 15 y 16 del notebook.

La comprobación de mayor valor es la última. El pipeline de clases se ejecuta sobre el archivo original, su salida se escribe como CSV en memoria y se compara con `F2/data/processed/yrbs2023_seleccion_procesada.csv` de dos formas: con `assert_frame_equal`, que verifica columnas, orden, tipos y valores (The pandas development team, s. f.), y comparando el texto completo del archivo. Ambas coinciden. Esto demuestra que trasladar las clases a módulos no alteró ni un valor, y por eso el notebook no guarda un nuevo conjunto de datos: el de la Fase 2 sigue siendo el oficial.

La misma lógica de dos caminos independientes se aplicó al núcleo analítico: el primer nivel del árbol de segmentos (sección 14) se calcula filtrando posiciones con NumPy, y la tabla de contingencia (sección 13) contando pares con `bincount`; ambos entregaron los mismos ocho porcentajes. Además,

antes de medir, la utilidad `comparar()` exige que todas las implementaciones den el mismo resultado y lanza `ValueError` si no coinciden. La trazabilidad del proceso queda en las salidas de cada celda, en la bitácora que el pipeline genera al ejecutarse (Observer) y en los diez archivos de F3/resultados/, que se releen tras guardarse para confirmar que se pueden auditar sin volver a ejecutar el cuaderno.

Manejo de excepciones y casos límite de las clases base. Además del pipeline real, se probó cómo reacciona la clase base `Transformador` y sus clases hijas ante situaciones extremas o usos incorrectos (sección 7 del notebook). La Tabla 5 resume cada caso y la reacción observada.

Tabla 5. Reacción del código ante casos límite y usos incorrectos

Caso	Prueba realizada	Reacción del código
Límite	Columna sin valores nulos ([2, 5, 8]) pasada por <code>ImputadorMediana</code>	Sin cambios: los tres valores se conservan
Límite	Columna con varianza cero ([4, 4, 4]) pasada por <code>EscaladorEstandar</code>	Devuelve [0, 0, 0]: la desviación cero se reemplaza por 1 y no hay división por cero
Límite	Una sola categoría en <code>CodificadorNominal</code>	Crea una sola columna, sin error
Límite	Categoría que aparece solo en prueba (código 9 en <code>sexo_cod</code>)	No crea columna nueva: el vocabulario se aprende en el ajuste y la categoría no vista se ignora
Excepción	Llamar a <code>.transformar()</code> antes de <code>.ajustar()</code> , en un paso o en el Pipeline	<code>RuntimeError</code> : «hay que ajustar antes de transformar»
Excepción	Asignar desde fuera una propiedad de solo lectura (ajustado)	<code>AttributeError</code> : la propiedad no tiene setter
Excepción	Ajustar sobre una columna inexistente	<code>KeyError</code> con el nombre del paso y de la columna
Excepción	Agregar al Pipeline un objeto que no es <code>Transformador</code>	<code>TypeError</code> : se esperaba un <code>Transformador</code>

Nota. Elaboración propia a partir de las secciones 3 y 7 del notebook.

La diferencia entre los dos codificadores es deliberada. El `CodificadorNominal` didáctico ignora una categoría que no vio en el entrenamiento, porque ningún modelo pudo aprender de ella. El `CodificadorOneHot` del proyecto, en cambio, rechaza con `ValueError` cualquier código que no esté en el *codebook* (Anexo C), porque en el YRBS un código no declarado indica un error del archivo y no una

categoría nueva. En ambos casos la reacción es explícita: el código nunca produce en silencio un resultado incorrecto.

d. Eficiencia y optimización

Las mediciones siguen las tres reglas acordadas en el foro de la Formativa 2 y en el material de la Semana 2: repetir cada medición y conservar el tiempo **mínimo**, que es el menos afectado por interrupciones del sistema operativo; medir en el tamaño real del conjunto; y comprobar antes que las versiones entregan el mismo resultado, porque una versión más rápida que da otra respuesta no es una optimización sino un error. El tiempo se midió con `timeit.repeat` y la memoria pico con `tracemalloc` (Python Software Foundation [PSF], s. f.-d, s. f.-e). Se compararon cuatro cálculos del proyecto; la Tabla 9 resume la decisión de cada uno.

d.1 Conteo de faltantes por fila (paso `MarcadorFaltantes`)

`MarcadorFaltantes` cuenta, para cada uno de los 20.103 estudiantes, cuántas de las 7 variables de análisis están vacías. Las tres implementaciones tienen la misma complejidad temporal teórica, $O(n \cdot k)$ con $k = 7$, porque todas miran cada celda una vez (Cormen et al., 2022); lo que cambia es el costo por celda.

Tabla 6. Conteo de faltantes por fila en el tamaño real ($n = 20.103$)

Implementación	Cómo trabaja	Tiempo mínimo (ms)	Memoria pico (MB)	Veces más lenta
con_bucle	<code>itertuples()</code> y conteo en Python	30,92	1,252	16,5
con_apply	<code>apply(..., axis=1)</code> : una llamada de Python y un <code>Series</code> por fila	487,18	3,437	260,5
vectorizada	<code>isna().sum(axis=1)</code> sobre la tabla completa, en C	1,87	1,212	1,0

Nota. Mínimo de 5 repeticiones. Fuente: F3/resultados/eficiencia_faltantes_f3.csv.

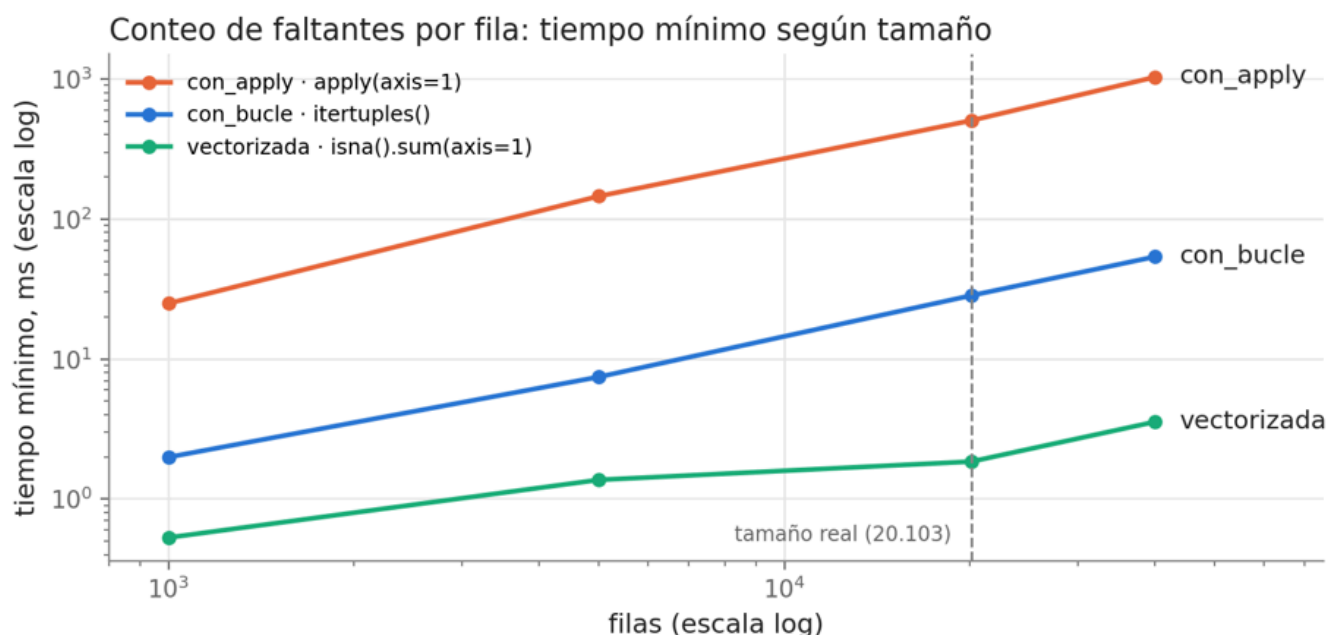


Figura 1. Tiempo mínimo de las tres implementaciones según tamaño del conjunto

Nota. Escala logarítmica en ambos ejes; la línea discontinua marca el tamaño real. Mínimo de 3 repeticiones por punto; valores en el Anexo A.

Interpretación. Las tres versiones crecen con el tamaño, como corresponde a $O(n \cdot k)$: entre 5.000 y 40.000 filas (8 veces más datos), `con_bucle` pasa de 7,45 a 53,84 ms y `con_apply` de 145,61 a 1.033,18 ms, cerca de 7 veces en ambos casos. La versión vectorizada crece menos (de 1,37 a 3,56 ms) porque parte de su costo es fijo; por eso la razón entre `apply` y la vectorizada no es constante (47 veces con 1.000 filas, 273 con 20.103). El resultado más relevante es que `apply(axis=1)` parece vectorizado pero no lo es: es la más lenta de las tres, unas 16 veces más que el propio bucle, porque además de llamar a una función de Python por fila construye un objeto `Series` para cada una (McKinney, 2022). **Complejidad espacial:** la vectorizada crea una matriz temporal de verdadero/falso de $n \times 7$ celdas, $O(n \cdot k)$ (del orden de 140 KB en el tamaño real), y aun así su pico (1,212 MB) es equivalente al del bucle (1,252 MB); `con_apply` lo triplica (3,437 MB). **Elección:** `MarcadorFaltantes` usa la versión vectorizada. La medición cambió una decisión: `apply` era la forma que el equipo habría escrito por intuición.

d.2 Núcleo del avance formativo, re-medido en el notebook central

Los scripts de la Formativa 3 se importan desde F3/src/ y se vuelven a medir en la misma ejecución con `comparar()` (Tabla 7). La tabla de contingencia $q80 \times q84$ confirma que el costo está en crear objetos de pandas por fila: `iterrows` es unas 2.400 veces más lenta que `bincount`, que codifica cada par como un índice entero y cuenta en una sola pasada compilada. En la búsqueda de 1.000 registros, la binaria recursiva (profundidad $\lceil \log_2 20.103 \rceil = 15$) es unas 240 veces más rápida que la lineal, y el diccionario, $O(1)$ por consulta, unas 22 veces más rápido que la binaria; a cambio, el diccionario ocupa 1,70 MB frente a 0,15 MB del arreglo ordenado, unas 11 veces más memoria. Como las auditorías consultan muchos registros sobre el mismo conjunto, se adopta el diccionario.

Tabla 7. Implementaciones del núcleo formativo medidas en el notebook central

Cálculo	Implementación	Complejidad	Tiempo mínimo (ms)	Memoria pico (KiB)
Contingencia $q80 \times q84$ (11.602 pares)	<code>bincount</code>	$O(n)$	0,070	454,0
	zip sobre arreglos	$O(n)$	1,824	2,7
	<code>crosstab</code>	$O(n)$	5,385	877,6
	<code>iterrows</code>	$O(n)$	168,197	50,2
1.000 búsquedas por <code>id_registro</code>	diccionario	$O(1)$ por consulta	0,124	8,9
	binaria recursiva	$O(\log n)$	2,675	40,4
	lineal	$O(n)$	637,763	36,8

Nota. Mínimo de 5 (contingencia) y 3 (búsqueda) repeticiones. La memoria de la búsqueda excluye la construcción del índice (1,70 MB). Fuentes: F3/resultados/eficiencia_contingencia_f3.csv y eficiencia_busqueda_f3.csv.

d.3 Recursión con memoización: prevalencia por segmentos anidados

El problema. La contingencia muestra que la proporción de salud mental no buena sube de 20,3 % entre quienes no usan redes a 35,3 % entre quienes las usan más de una vez por hora. La pregunta siguiente es si ese gradiente se mantiene dentro de cada sexo y edad. `ArbolPrevalencia` (F3/src/segmentacion.py) desciende por la jerarquía uso de redes $\rightarrow$ sexo $\rightarrow$ edad y calcula en cada segmento el porcentaje con salud mental no buena. Un segmento solo se subdivide si tiene al menos 30 estudiantes; por esa **poda**, la profundidad de cada rama depende de los datos y no se conoce al escribir

el código, que es el criterio del proyecto para usar recursión (bitácora F3.7).

Dónde se repite trabajo. El subconjunto de un segmento se define de forma recursiva: es el subconjunto de su padre filtrado por un valor más. Sin memoria, cada nodo vuelve a calcular a todos sus ancestros desde las 15.705 filas, el mismo defecto de Fibonacci ingenuo; con `functools.lru_cache` (PSF, s. f.-b) cada segmento se calcula una sola vez y cada hijo filtra solo el arreglo, ya calculado, de su padre:

```
def _subconjunto_sin_memoria(self, filtros):
    if not filtros:
        return self._todos
    padre = self._subconjunto(filtros[:-1])
    columna, valor = filtros[-1]
    return padre[self._x[columna][padre] == valor]

# en __init__: la misma función, envuelta o no con lru_cache
self._subconjunto = (lru_cache(maxsize=None)(self._subconjunto_sin_memoria)
                     if memoizar else self._subconjunto_sin_memoria)
```

Se compararon tres implementaciones que entregan la misma tabla de 121 segmentos (comprobado con `assert_frame_equal`): la recursiva ingenua, $O(\text{nodos} \cdot n)$; la recursiva memoizada, $O(\text{profundidad} \cdot n)$; y una alternativa vectorizada con un `groupby` por nivel y la poda aplicada después. Además del tiempo, se contó el trabajo: cuántas veces se filtró un arreglo (Tabla 8 y Figura 2).

Tabla 8. Segmentos por profundidad de la jerarquía: trabajo, tiempo y memoria ($n = 15.705$)

Niveles	Segmentos	Filtros ingenua	Filtros memoizada	Ingenua (ms)	Memoizada (ms)	groupby (ms)	Ingenua / memoizada
1	9	8	8	2,87	3,17	8,16	0,9
2	25	48	24	5,26	5,57	13,59	0,9
3	121	368	120	14,90	7,51	18,66	2,0
4	530	2.187	529	54,77	12,11	30,87	4,5

Nota. Jerarquía redes → sexo → edad → sueño; se subdivide un segmento con $n \geq 30$. Mínimo de 9 repeticiones. Memoria pico con `tracemalloc`: $\approx 4,5$ – $4,8$ MB en ambas recursiones y 11,4 MB con `groupby`. Fuente:

F3/resultados/eficiencia_memoizacion_f3.csv.

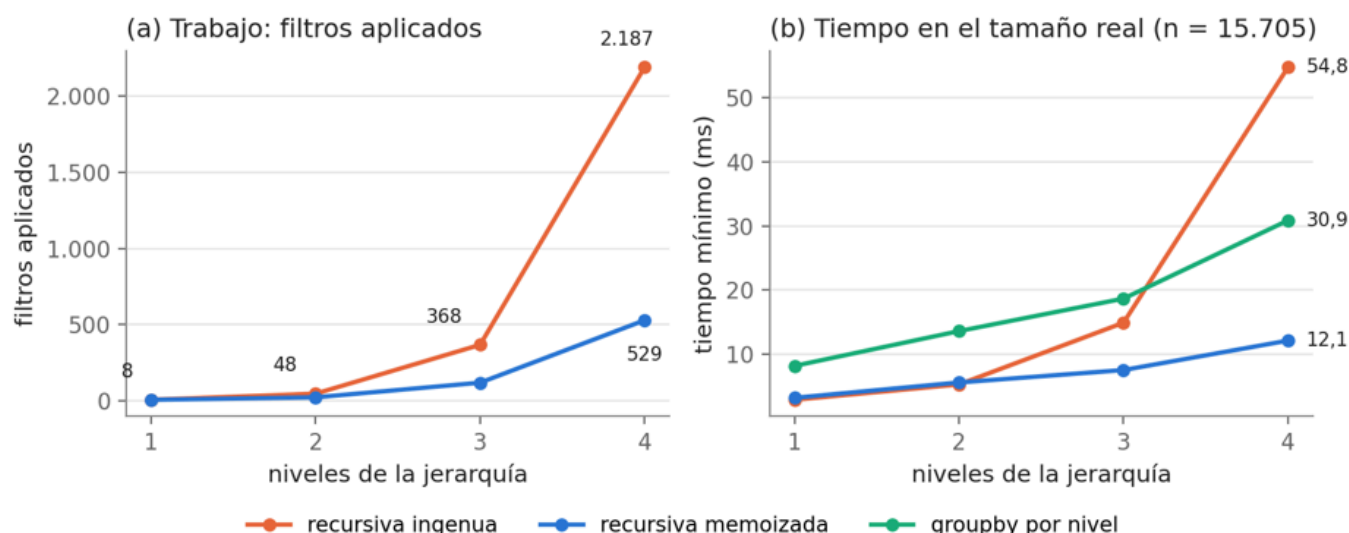


Figura 2. Trabajo y tiempo de las tres implementaciones según la profundidad

Nota. (a) Filtros aplicados; (b) tiempo mínimo en el tamaño real. Elaboración propia a partir de la sección 14 del notebook.

Interpretación. Con un nivel no hay ancestros que repetir: ambas recursiones aplican los mismos 8 filtros y la memoizada es algo más lenta (razón 0,9) por el costo de administrar la caché; se reporta porque muestra que la memoización no es gratis. Desde dos niveles la ingenua repite el trabajo de los ancestros en cada nodo: sus filtros crecen como $\text{nodos} \times \text{profundidad}$ (2.187 con 4 niveles), mientras la memoizada aplica exactamente uno por segmento (529). Con dos niveles ese ahorro (24 filtros) todavía no compensa el costo de la caché y ambas quedan prácticamente empatadas (razón 0,9); desde tres niveles la razón de tiempo crece con cada nivel: 2,0 veces con tres y 4,5 con cuatro. La memoizada supera también al groupby (12,11 frente a 30,87 ms con 4 niveles): aunque vectorizado, groupby reagrupa las 15.705 filas en cada nivel y descarta después los segmentos podados, mientras la recursión solo trabaja dentro de los segmentos que sobreviven y cada hijo filtra un arreglo cada vez más pequeño.

Costo espacial: la caché retiene las posiciones de cada segmento, $O(\text{profundidad} \cdot n)$ enteros, que en el tamaño real son 0,47 MB; el pico de `tracemalloc` no cambia entre las recursiones porque lo domina la preparación de los arreglos, y groupby tiene el mayor pico por sus tablas intermedias.

Elección final. Para la descripción por segmentos se adopta la **recursión memoizada**: entrega la misma tabla que las otras dos, es la más rápida desde tres niveles, que es la profundidad de `JERARQUIA_SEGMENTOS`, respeta la poda sin calcular segmentos que luego se descartan y cuesta

menos de un megabyte de memoria. Este caso reemplaza al ejemplo de Fibonacci de la Formativa 3, como sugirió la retroalimentación.

Lo que muestra el resultado. El gradiente por uso de redes aparece en ambos sexos, pero con pendientes muy distintas (Figura 3): en el sexo femenino la proporción de salud mental no buena sube de 27,3 % (no usa) a 45,8 % (más de una vez por hora); en el masculino, de 16,3 % a 22,5 %, y la brecha crece con la frecuencia de uso. El patrón es coherente con lo reportado por el CDC con el mismo levantamiento, ponderado: la tristeza o desesperanza persistente es más frecuente entre usuarios frecuentes de redes, con la mayor diferencia en las estudiantes (Young et al., 2024). Aquí el resultado es muestral y sin ponderar; los segmentos de nivel 3 con menos de 30 estudiantes (35 de 96) se reportan, pero no se interpretan.

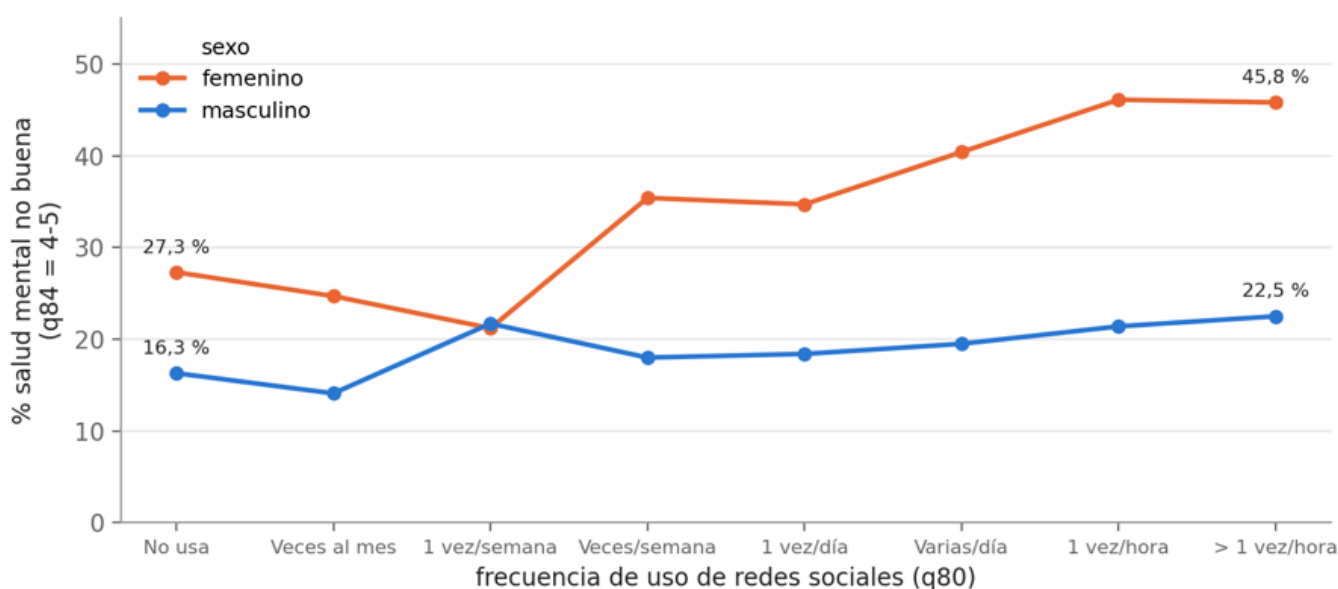


Figura 3. Porcentaje con salud mental no buena por frecuencia de uso de redes y sexo

Nota. q84 = 4–5 («la mayor parte del tiempo» o «siempre»). Muestral, sin ponderar; n = 15.705 con el desenlace respondido. Valores en el Anexo E.

d.4 Síntesis de las decisiones de eficiencia

Tabla 9. Algoritmo adoptado en cada cálculo, con su justificación temporal y espacial

Cálculo	Adoptado	Tiempo	Espacio	Descartado y por qué
Faltantes por fila	vectorizada	16,5× más rápida que el bucle	Pico igual al bucle (≈1,2 MB)	apply: 260× más lenta y 3× la memoria
Contingencia q80 × q84	bincount	≈2.400× más rápida que iterrows	O(8·5) celdas + arreglo temporal (454 KiB)	crosstab: 77× más lenta y el mayor pico
Búsqueda por id	diccionario	22× más rápido que la binaria	≈11× el arreglo (1,70 MB)	lineal: O(n) por consulta
Prevalencia por segmentos	recursión memoizada	4,5× la ingenua y 2,5× groupby (4 niveles)	Caché de 0,47 MB	ingenua: repite ancestros; groupby: 11,4 MB de pico
Clasificar horas de sueño (didáctico)	pd.cut	2,8× con 50.000 filas; más lenta con 1.000	3× la memoria (0,51 frente a 0,17 MB)	bucle: gana solo en conjuntos pequeños

Nota. Elaboración propia a partir de las secciones 9, 12, 13 y 14 del notebook. Los tiempos absolutos dependen del equipo; lo reproducible son las razones entre implementaciones y la forma de las curvas.

e. Diseño estructurado del código

La arquitectura se construyó para cumplir dos criterios verificables. **Alta cohesión:** cada clase y cada módulo hacen una sola cosa y su propósito cabe en una frase, que es la primera línea de su *docstring*; la sección 6.4 del notebook lo comprueba leyendo esa línea en los catorce módulos. La sección 6 lo contrasta con un contraejemplo, `ProcesadorTodoEnUno`, que carga, limpia, grafica y envía correos; al separarlo en `Cargador` y `Limpiador`, la limpieza se puede probar sin tocar el disco. **Bajo acoplamiento:** los componentes dependen de una interfaz, no de otras clases concretas. La prueba práctica propuesta en la guía —si se cambia la forma de tratar los faltantes, ¿cuántos archivos hay que modificar?— tiene respuesta uno: se cambia el argumento `estrategia` de `TratamientoFaltantes` y ni `Pipeline` ni los demás pasos se tocan. Del mismo modo, `ArbolPrevalencia` consume la salida del pipeline sin conocer sus pasos.

Flujo de datos. Es lineal y documentado: archivo original → 12 transformadores en orden

declarado $\rightarrow$ DataFrame $20.103 \times 26 \rightarrow$ verificación contra F2 $\rightarrow$ núcleo analítico (contingencia y segmentos) $\rightarrow$ registro en F3/resultados/. Cada transformador recibe una copia del DataFrame y devuelve otra, de modo que ningún paso modifica la entrada de otro sin que quede registrado.

Recursividad o división funcional. El criterio adoptado en la Formativa 3 y consignado en el Apunte de la Fase 3 es que la recursión conviene cuando la profundidad del problema no se conoce de antemano. Se usa en tres lugares: `aplanar()`, que convierte metadatos anidados de profundidad variable en pares clave–valor; la búsqueda binaria, cuya profundidad es $\log_2 n$; y `ArbolPrevalencia.explorar()`, cuya profundidad depende de la poda. Los pasos de transformación recorren una lista fija de columnas y para eso se usó división funcional con bucles y operaciones vectorizadas. La decisión se apoya en la evidencia del avance formativo: una recursión que avanza fila por fila falla por el límite de profundidad de Python sobre unas 1.000 filas, y la versión *divide y vencerás* funciona, pero resultó más lenta que el bucle equivalente.

Escalabilidad. Agregar un paso nuevo, por ejemplo, otro indicador, exige una línea en la fábrica `construir_pipeline_proyecto` y ninguna modificación en `Pipeline`. Una nueva estrategia de faltantes es una clase hija de `EstrategiaFaltantes` con un único método, `tratar`. Agregar una variable a la segmentación es agregarla a `JERARQUIA_SEGMENTOS`: la recursión y la caché se adaptan solas. Para la Fase 4, el análisis ponderado con `weight`, `stratum` y `psu` podrá agregarse como un módulo nuevo que reciba la salida del pipeline, sin reescribir el código existente.

III. Implementación de código modular y robusto

a. Programación orientada a objetos

El núcleo algorítmico es una jerarquía de clases con una clase base, Transformador, que define el contrato de todo paso del pipeline y controla su estado. Las clases hijas solo implementan lo que cambia de un paso a otro: qué aprenden (aprender) y cómo lo aplican (aplicar). Tras la retroalimentación de la Formativa 3, ninguna clase se define en el notebook: todas viven en los archivos de F3/src/ que proyectó la Tabla 8 de ese informe (transformadores.py, estrategias.py, pipeline.py, fabrica.py, observadores.py y carga.py; los ejemplos de las secciones 2, 3 y 6, en didacticos.py) y el notebook las importa (Figura 4). La Tabla 10 señala dónde se aplica cada principio (PSF, s. f.-a).

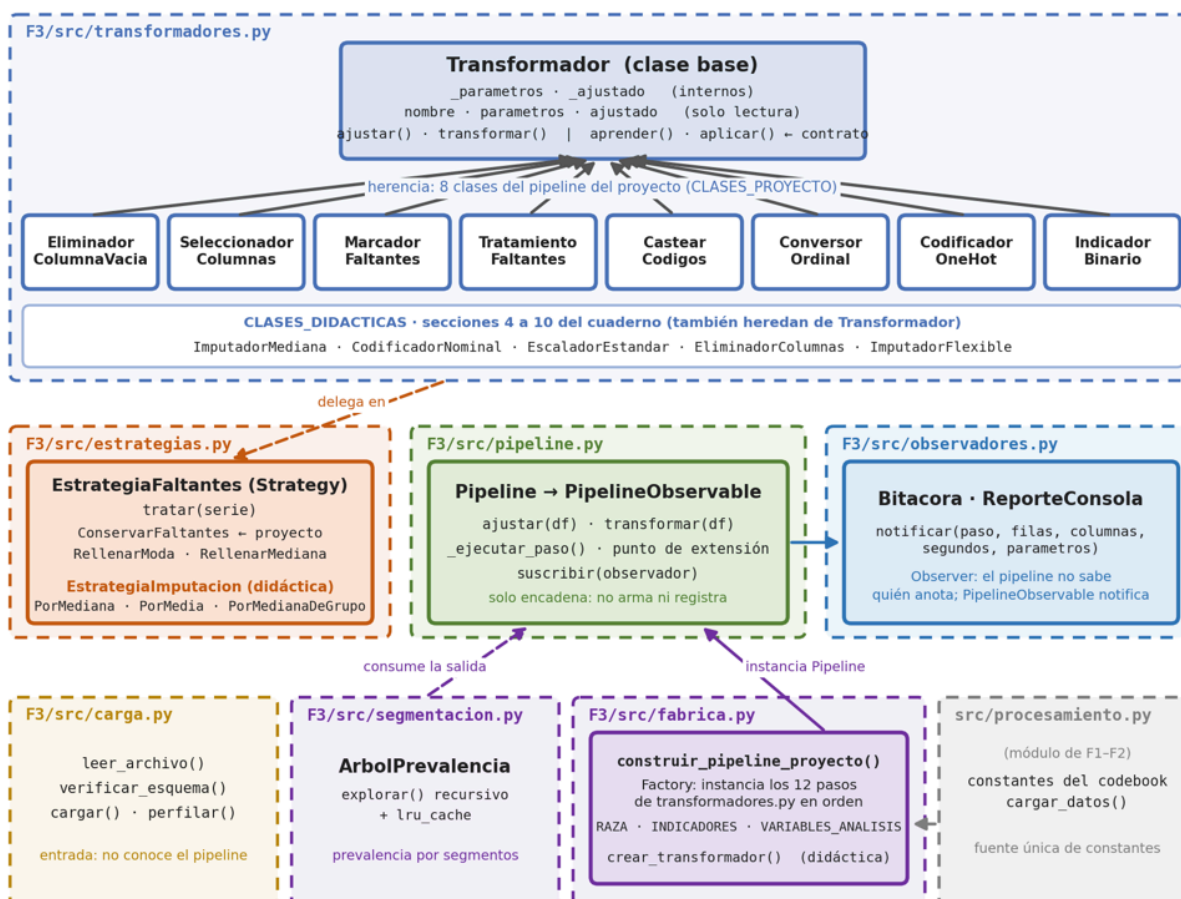


Figura 4. Diagrama de clases del núcleo algorítmico, agrupado por módulo

Nota. Elaboración propia a partir de F3/src/ y de la sección 17 del notebook.

Tabla 10. Principios de la programación orientada a objetos en el código del proyecto

Principio	Dónde se aplica	Qué evita
Encapsulamiento	<code>_parametros</code> y <code>_ajustado</code> son internos; <code>nombre</code> , <code>parametros (copia)</code> y <code>ajustado</code> son propiedades de solo lectura; <code>transformar()</code> falla si no se llamó antes a <code>ajustar()</code> ; <code>_codigos()</code> , <code>_ejecutar_paso()</code> y la caché de <code>ArbolPrevalencia</code> son internos	Que el estado se modifique desde fuera y que un uso en orden incorrecto produzca un resultado sin sentido y sin error
Herencia	8 clases del proyecto heredan de <code>Transformador</code> ; <code>PipelineObservable</code> hereda de <code>Pipeline</code> y solo redefine <code>_ejecutar_paso()</code> ; 3 estrategias heredan de <code>EstrategiaFaltantes</code> . Se verificó con <code>isinstance</code> y con la cadena <code>__mro__</code>	Repetir en cada paso la validación de columnas, el control de estado, la copia defensiva y el recorrido de los pasos
Polimorfismo	<code>Pipeline</code> llama <code>ajustar/transformar</code> sin preguntar el tipo de cada paso; <code>TratamientoFaltantes</code> llama <code>tratar()</code> sin saber qué estrategia recibió; las hijas que trabajan con varias columnas redefinen <code>columnas_requeridas()</code>	Modificar el orquestador cada vez que se agrega un paso o una estrategia

Nota. Elaboración propia a partir de la sección «Cierre» del notebook.

El diseño de `Transformador` sigue el patrón *Template Method* (Gamma et al., 1994): la clase base fija el orden de validar, aprender y marcar el estado, y cada clase hija completa solo lo que le es propio (`aprender()` y `aplicar()`). Por eso agregar un transformador nuevo no exige tocar `Pipeline`, cumpliendo el principio abierto/cerrado de SOLID: el sistema queda abierto a extensión y cerrado a modificación. En el siguiente fragmento se aprecia la clase `Transformador` con sus hijas:

```

class Transformador:
    def ajustar(self, df):
        faltan = [c for c in self.columnas_requeridas() if c not in df.columns]
        if faltan:
            raise KeyError(f"{self.nombre}: no existen las columnas {faltan}.")
        self._parametros = self.aprender(df)
        self._ajustado = True
        return self

    def transformar(self, df):
        if not self._ajustado:
            raise RuntimeError(f"{self.nombre}: hay que ajustar antes de transformar.")
        return self.aplicar(df.copy())

class IndicadorBinario(Transformador):
    def aplicar(self, df):
        codigos = self._codigos(df)
        indicador = codigos.isin(self.valores_positivos).astype("Int8")
        indicador[codigos.isna()] = pd.NA
        df[self.nuevo_nombre] = indicador
        return df

```

Encapsulamiento y prevención de fuga de datos. Separar `ajustar()` de `transformar()`

evita que las estadísticas usadas para transformar un dato se calculen mirando también la partición de prueba. `pipeline.ajustar(df_train)` fija los `_parametros` únicamente desde el entrenamiento; `pipeline.transformar(df_test)` aplica esos mismos parámetros sin recalcular nada sobre la prueba. La cifra que lo demuestra ya se reportó en el apartado II.b: la media de `edad_cod` escalada en el conjunto de prueba es $-0,013$ y no cero, justo lo esperable si los parámetros vienen solo del entrenamiento.

Las clases cumplen el criterio de la guía para justificar una clase: **recuerdan algo entre llamadas**. Cada transformador guarda lo aprendido en `_parametros` —por ejemplo, el porcentaje de positivos de cada indicador o el número de casos completos—, `Pipeline` recuerda si ya fue ajustado y `ArbolPrevalencia` recuerda los subconjuntos ya calculados. Las piezas que no necesitan recordar nada (comparar, aplanar, las funciones de contingencia y búsqueda) siguen siendo funciones. Todos los métodos tienen *docstring*, y la tabla de arquitectura del apartado siguiente se genera leyendo esas *docstrings*, lo que obliga a mantenerlas.

b. Documentación de arquitectura y funcionalidad del código

La tabla de componentes se genera automáticamente en la sección 17 del notebook: `__subclasses__()` devuelve las clases que realmente heredan de cada base, e `inspect.getfile` informa en qué archivo está definida cada una (PSF, s. f.-c). Así la documentación refleja el código ejecutado y no puede quedar desactualizada: si una clase se mueve de archivo, la tabla cambia en la siguiente ejecución. Se guarda en `F3/resultados/arquitectura_f3.csv` (40 componentes: 25 del proyecto y 15 didácticos, ninguno definido en el cuaderno) y su versión resumida es la Tabla 11. La misma sección compara la Tabla 8 de la Formativa 3 con el archivo real de cada componente: los 15 componentes planificados están en el archivo proyectado y la celda falla si alguno queda fuera (`F3/resultados/tabla8_ejecutada_f3.csv`).

Tabla 11. Componentes del núcleo algorítmico por archivo, generados desde el código

Archivo	Componentes	Rol en la arquitectura
<code>F3/src/transformadores.py</code>	Transformador y sus 8 clases hijas	Pasos del pipeline: cambian si cambia una decisión de datos
<code>F3/src/estrategias.py</code>	EstrategiaFaltantes y 3 estrategias	Strategy: la parte que el equipo prevé modificar en la Fase 4
<code>F3/src/pipeline.py</code>	Pipeline, PipelineObservable, Bitacora, ReporteConsola, construir_pipeline_proyecto	Orquestación, Observer y Factory: cambian solo si cambia la forma de encadenar
<code>F3/src/segmentacion.py</code>	ArbolPrevalencia, tabla_por_niveles_groupby	Núcleo analítico: recursión con memoización y su alternativa
<code>F3/src/contingencia.py</code> , <code>analisis.py</code> , <code>busqueda.py</code> , <code>recursion.py</code>	Funciones y AnalisisContingencia de la Formativa 3	Núcleo analítico integrado sin copiarlo
<code>F3/src/medicion.py</code>	medir, comparar	Utilidad independiente del proyecto: mide cualquier función
<code>src/procesamiento.py</code>	Constantes del <i>codebook</i> , cargar_datos	Módulo de F1–F2: fuente única de las constantes
cuaderno (didáctico)	5 clases de ejemplo (ImputadorMediana, EscaladorEstandar, ...)	Explicar conceptos; no forman parte del pipeline

Nota. Elaboración propia a partir de la sección 17 del notebook y de `F3/resultados/arquitectura_f3.csv`.

Patrón de diseño principal: Strategy. En la Fase 2 se compararon cuatro formas de tratar los

faltantes y cada una era un bloque de código distinto; cambiar de decisión obligaba a reescribir el pipeline. Con `Strategy`, `TratamientoFaltantes` sabe *cuándo* tratar los faltantes y delega *cómo* hacerlo en una estrategia intercambiable que se elige al construir el pipeline. Gracias a eso, la comparación de la Tabla 3 se obtuvo cambiando solo un argumento, y si en la Fase 4 se decidiera imputar, bastaría con pasar otra estrategia sin arriesgar los demás pasos (Gamma et al., 1994).

```
class ConservarFaltantes(EstrategiaFaltantes):    # F3/src/estrategias.py

    """Estrategia del proyecto (bitácora 2.5): los faltantes se conservan como NA."""

    etiqueta = "conservar NA"

    def tratar(self, serie):

        return serie                                # la decisión queda explícita

# Cambiar la decisión es cambiar un argumento, no el pipeline
construir_pipeline_proyecto(estrategia=RellenarModa())
```

Patrones de apoyo. `Factory` (`construir_pipeline_proyecto`, en `fabrica.py`) concentra en un solo lugar la decisión de qué transformador construir para cada variable, a partir de las constantes de `src/procesamiento.py`. `Observer` (`PipelineObservable`, en `pipeline.py`, y `Bitacora`, en `observadores.py`) registra filas, columnas, tiempo y parámetros de cada paso sin que el pipeline sepa quién anota: `PipelineObservable` hereda el recorrido de `Pipeline` y solo redefine el punto de extensión `_ejecutar_paso()`. Se descartó `Singleton`: la configuración ya vive en una sola celda y en las constantes del módulo de F2, y agregar estado global complicaría las pruebas sin resolver un problema real del proyecto.

Dos carpetas de código, con propósitos declarados. `src/procesamiento.py`, en la raíz, es el módulo de F1–F2: contiene las constantes del *codebook* y las funciones de carga que usan los notebooks de esas fases; sigue vigente como fuente única de las constantes. `F3/src/` contiene el núcleo algorítmico de F3 y lo usa el notebook central; `fabrica.py` importa las constantes de `src/` en vez de copiarlas. No se consolidaron en una sola carpeta porque tienen propósitos distintos y hacerlo obligaría a modificar notebooks ya evaluados (bitácora F3.20). **El README lo declara en una sección propia.**

Registro ordenado del avance. La evolución del código y el razonamiento detrás de cada cambio quedan en `docs/bitacora_decisiones.md`, con la cifra que respalda cada decisión. La Tabla 12 resume cómo se incorporó la retroalimentación de la Formativa 3.

Tabla 12. Retroalimentación de la Formativa 3 y cambio incorporado

Observación del docente	Cambio en esta entrega	Bitácora
Dos notebooks en F3/notebooks/: no se sabe cuál evaluar	Un solo notebook central; el de la Formativa pasa a F3/notebooks/formativa3/ como registro histórico	F3.21, 3.10
Dos carpetas src: declarar cuál está vigente o consolidarlas	Clases trasladadas a F3/src/; README con la sección «Dos carpetas src» y el propósito de cada una	F3.19, F3.20, 3.11
Fibonacci ilustra bien, pero no es un problema del YRBS	Memoización aplicada a la prevalencia por segmentos (d.3): 4,5 veces más rápida que la recursión ingenua	F3.23, F3.24

Nota. Elaboración propia a partir de la rúbrica comentada de la Formativa 3 y de `docs/bitacora_decisiones.md`.

Evolución desde la Fase 2. Antes, el pipeline era una secuencia de celdas que llamaban funciones sueltas, con el orden implícito en la posición de cada celda. En el avance formativo, las funciones salieron a módulos y se midió su costo. Ahora, el mismo trabajo se declara como una lista de objetos con nombre, que viven en módulos, con el orden explícito en un solo lugar y verificado contra el resultado anterior. Las decisiones F3 que cierran esta fase son: (1) el pipeline de F2 se reorganiza en clases sin cambiar su resultado; (2) los faltantes se conservan como decisión explícita en el código; (3) cada cálculo usa la implementación que ganó la medición (Tabla 9); (4) la recursión se usa solo donde la profundidad es desconocida y, donde además repite trabajo, se memoriza.

IV. Repositorio GitHub (F3)

El repositorio del proyecto (<https://github.com/MatiasManriquezO/proyecto-grupo8-mcdi500>) conserva las carpetas F1 y F2 —la salida de F2 es la entrada de F3—, mantiene la carpeta F4, creada desde el inicio del proyecto para la fase siguiente, y organiza la carpeta F3 con un solo notebook central, el núcleo algorítmico en F3/src/ y los resultados que respaldan este informe:

```

proyecto-grupo8-mcdi500/
├── src/procesamiento.py           módulo de F1-F2 (constantes, carga)
├── F1/ data/raw/ · notebooks/    archivo original del CDC
├── F2/ data/processed/ · notebooks/ salida de F2 = entrada de F3
├── F3/
│   ├── notebooks/
│   │   ├── S2_F3_NucleoAlgoritmico_P00_Grupo8.ipynb ★ notebook central
│   │   └── formativa3/          registro histórico
│   ├── src/ carga · transformadores · estrategias · pipeline · fabrica ·
│   │   observadores · segmentacion · contingencia · analisis ·
│   │   busqueda · recursion · medicion · didacticos
│   ├── data/_demo/              salidas de la Formativa 3
│   └── resultados/ *_f3.csv (Sumativa 2) · tabla*.csv (Formativa 3)
├── F4/ notebooks/Fase 4.md       creada desde el inicio: análisis de la Fase 4
├── docs/ bitacora_decisiones.md · Informe/ · diccionario_variables.md
├── .mailmap · .gitignore · requirements.txt
└── README.md                    ejecución, dependencias y organización

```

El **README técnico** describe la estructura de la Fase 3, las instrucciones de ejecución del notebook central (qué archivos lee, qué produce y qué sección verifica contra F2), el entorno de la ejecución versionada, las dependencias —no se requieren librerías nuevas: `timeit`, `tracemalloc`, `functools` e `inspect` pertenecen a la biblioteca estándar—, la sección «Dos carpetas src» y una tabla de los módulos de F3/src/ con el criterio de optimización aplicado en cada uno.

Control de versiones y contribuciones. Los cambios se incorporan con la convención de commits del proyecto (`feat`, `fix`, `test`, `docs`, `data`), con las cuentas universitarias de los cuatro integrantes y un `.mailmap` que unifica sus identidades, de modo que `git shortlog -sne` muestra las contribuciones por persona. La Tabla 13 muestra la distribución del trabajo de esta entrega.

Tabla 13. Contribuciones de la Sumativa 2 por integrante

Integrante	Cantidad de commits de esta entrega
Roberto Sánchez Saldivia	14
Daniel Pérez Ramirez	18
Matías Manriquez Ortiz	9
Abigail Robles Chávez	3

Nota. Elaboración propia. El historial completo se consulta con `git log` en el repositorio.

V. Notebooks ejecutables (F3)

El notebook `F3/notebooks/S2_F3_NucleoAlgoritmico_P00_Grupo8.ipynb` es el **único entregable de la Fase 3**. Se ejecutó completo con Restart Kernel and Run All Cells: 116 celdas, 54 de código, numeración continua de 1 a 54, sin errores ni advertencias y con todas las salidas visibles (Python 3.13.15, pandas 3.0.6, NumPy 2.5.3, las versiones de `requirements.txt`). Cada bloque de código está precedido por una celda de texto que explica qué hace y seguido, cuando corresponde, por su interpretación, práctica recomendada para cuadernos reproducibles (Pimentel et al., 2021).

La estructura sigue el flujo de la fase: configuración, carga y perfil (sección 0); de funciones a objetos, encapsulamiento, herencia y Pipeline (1 a 5); cohesión y acoplamiento, con la estructura real de módulos (6); validación de las clases base (7); recursividad (8); medición (9); patrones de diseño (10); y el trabajo aplicado al proyecto: el pipeline de F2 en clases (11), eficiencia sobre un paso real (12), integración del avance formativo (13), recursión con memoización (14), validación del pipeline y del núcleo (15), Strategy y verificación contra F2 (16), y arquitectura, Tabla 8 ejecutada y registro (17). Las secciones 2 a 10 usan clases didácticas para explicar cada concepto; también se importan de `F3/src/` (`didacticos.py` y `CLASES_DIDACTICAS`) y se muestran con `inspect.getsource`. El código del proyecto se usa desde la sección 11.

La reproducibilidad descansa en cuatro elementos: rutas relativas a la raíz, localizada con `encontrar_raiz()` y declaradas en una sola celda; semilla fija (42); dependencias en el `requirements.txt` único de la raíz; y resultados guardados en `F3/resultados/` con verificación de ida y vuelta. La integración con las fases anteriores es explícita: el notebook importa `src/procesamiento.py` de la Fase 2, reconstruye su pipeline desde el archivo original de F1 y comprueba que produce exactamente el archivo de F2. La integración con el avance formativo se resume en la Tabla 14.

Tabla 14. Del avance formativo al notebook central

Resultado del avance formativo	Cómo se integra en esta entrega	Sección
Scripts de contingencia, búsqueda y AnalisisContingencia en F3/src/	Se importan sin copiarlos y se re-miden con <code>comparar()</code>	§13
Tres reglas de medición: mínimo, tamaño real, mismo resultado	Aplicadas a faltantes, contingencia, búsqueda y segmentos	§9, §12–§14
Recursión solo con profundidad desconocida	<code>aplanar()</code> , búsqueda binaria y <code>ArbolPrevalencia</code> con poda	§8, §13, §14
Fibonacci ingenuo frente a memoizado (ejemplo de manual)	Reemplazado por memoización en un cálculo del YRBS	§14
Clases definidas en el notebook	Trasladadas a los archivos de F3/src/ que proyectó la Tabla 8; el notebook no define ninguna clase y la sección 17 lo verifica	§4, §11, §17

Nota. Elaboración propia a partir del informe y la rúbrica comentada de la Formativa 3 y del notebook de esta entrega.

Como limitación, los tiempos absolutos dependen del equipo en que se ejecute el notebook; lo reproducible son las razones entre implementaciones y la forma de las curvas de crecimiento. Además, los resultados describen la muestra y no la población, porque el diseño muestral se incorporará en la Fase 4.

VI. Referencias

Centers for Disease Control and Prevention. (2024). *2023 Youth Risk Behavior Survey data* [Conjunto de datos]. <https://www.cdc.gov/yrbs/data/index.html>

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4.^a ed.). MIT Press.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

McKinney, W. (2022). *Python for data analysis: Data wrangling with pandas, NumPy, and Jupyter* (3.^a ed.). O'Reilly Media.

Pimentel, J. F., Murta, L., Braganholo, V., & Freire, J. (2021). Understanding and improving the quality and reproducibility of Jupyter notebooks. *Empirical Software Engineering*, 26(4), Artículo 65. <https://doi.org/10.1007/s10664-021-09961-9>

Python Software Foundation. (s. f.-a). *Classes*. <https://docs.python.org/3/tutorial/classes.html>

Python Software Foundation. (s. f.-b). *functools — Higher-order functions and operations on callable objects*. <https://docs.python.org/3/library/functools.html>

Python Software Foundation. (s. f.-c). *inspect — Inspect live objects*. <https://docs.python.org/3/library/inspect.html>

Python Software Foundation. (s. f.-d). *timeit — Measure execution time of small code snippets*. <https://docs.python.org/3/library/timeit.html>

Python Software Foundation. (s. f.-e). *tracemalloc — Trace memory allocations*. <https://docs.python.org/3/library/tracemalloc.html>

The pandas development team. (s. f.). *pandas.testing.assert_frame_equal*. https://pandas.pydata.org/docs/reference/api/pandas.testing.assert_frame_equal.html

Universidad Andrés Bello. (2026a). *Apunte de la Fase 3: Núcleo algorítmico, eficiencia y programación orientada a objetos* [Material docente]. MCDI500 Programación para la Ciencia de Datos.

Universidad Andrés Bello. (2026b). *Guía de apoyo: Evaluación Sumativa 2 (Fase 3). Núcleo algorítmico, eficiencia e implementación orientada a objetos* [Material docente]. MCDI500 Programación para la Ciencia de Datos.

Young, E., McCain, J. L., Mercado, M. C., Ballesteros, M. F., Moore, S., Licitis, L., Stinson, J., Jones, S. E., & Wilkins, N. J. (2024). Frequent social media use and experiences with bullying victimization, persistent feelings of sadness or hopelessness, and suicide risk among high school students — Youth Risk Behavior Survey, United States, 2023. *MMWR Supplements*, 73(4), 23–30. <https://doi.org/10.15585/mmwr.su7304a3>

Anexos

Anexo A. Conteo de faltantes por fila: tiempo mínimo (ms) según tamaño

n (filas)	con_bucle	con_apply	vectorizada	apply / vectorizada
1.000	1,99	24,97	0,53	47
5.000	7,45	145,61	1,37	106
20.103	28,41	505,28	1,85	273
40.000	53,84	1.033,18	3,56	290

Nota. Muestras con reemplazo cuando $n > 20.103$, solo para medir. Mínimo de 3 repeticiones. Fuente:

F3/resultados/eficiencia_crecimiento_f3.csv.

Anexo B. Bitácora de ejecución y parámetros aprendidos por el pipeline del proyecto

#	Paso	Columnas	Tiempo (s)	Parámetros aprendidos
1	EliminadorColumnaVacía(orig_rec)	116	0,0141	pct_nulos = 100,0
2	SeleccionadorColumnas	11	0,0036	116 columnas de entrada → 11 de salida
3	MarcadorFaltantes	13	0,0056	7 variables; 10.941 casos completos
4	TratamientoFaltantes	13	0,0011	estrategia «conservar NA»; 13.813 nulos
5	CastearCodigos	13	0,0035	7 columnas a Int64
6	ConversorOrdinal	13	0,0026	5 escalas ordinales
7	CodificadorOneHot(raceeth_cod)	21	0,0053	8 columnas de raza/etnicidad
8	IndicadorBinario(sexo_cod)	22	0,0059	positivos [1]; 49,6 %
9	IndicadorBinario(salud_mental_cod)	23	0,0059	positivos [4, 5]; 30,6 %
10	IndicadorBinario(redes_sociales_cod)	24	0,0064	positivos [6, 7, 8]; 78,1 %
11	IndicadorBinario(sueno_cod)	25	0,0062	positivos [5, 6, 7]; 23,0 %
12	IndicadorBinario(actividad_fisica_cod)	26	0,0067	positivos [6, 7, 8]; 48,0 %

Nota. 20.103 filas en todos los pasos. Fuente: F3/resultados/bitacora_ejecucion_f3.csv y parametros_pipeline_f3.csv.

Anexo C. Pruebas de excepción del pipeline real y del núcleo analítico

Entrada alterada	Excepción esperada	Componente que la detecta
Código de raza fuera del <i>codebook</i> (9)	ValueError	CodificadorOneHot
Código no entero (q84 = 2,5)	ValueError	CastearCodigos
orig_rec con un dato	ValueError	EliminadorColumnaVacía
Archivo sin la columna q85	KeyError	SeleccionadorColumnas
record repetido	ValueError	SeleccionadorColumnas
Transformar sin ajustar	RuntimeError	Pipeline
Indicador sin valores positivos	ValueError	IndicadorBinario (constructor)
Columna inexistente en la jerarquía	KeyError	ArbolPrevalencia
n_mínimo igual a cero	ValueError	ArbolPrevalencia
Estrategia que no es EstrategiaFaltantes	TypeError	TratamientoFaltantes
Observador sin método notificar()	TypeError	PipelineObservable
Versiones comparadas que no coinciden	ValueError	comparar()

Nota. Las 12 pruebas lanzaron la excepción esperada. Elaboración propia a partir de la sección 15 del notebook.

Anexo D. Memoria pico medida en la comparación de segmentos (MB)

Niveles	Recursiva ingenua	Recursiva memoizada	groupby por nivel
1	4,483	4,483	11,424
2	4,604	4,604	11,424
3	4,724	4,724	11,424
4	4,844	4,844	11,424

Nota. Pico con `tracemalloc` durante una ejecución completa, incluida la preparación de los arreglos. La caché de la versión memoizada retiene 0,47 MB con 4 niveles. Fuente: F3/resultados/eficiencia_memoizacion_f3.csv.

Anexo E. Porcentaje con salud mental no buena por uso de redes y sexo

Uso de redes sociales (q80)	Total	Femenino	Masculino	Diferencia (pp)
1 · No usa	20,3	27,3	16,3	11,0
2 · Unas veces al mes	17,6	24,7	14,1	10,6
3 · Una vez a la semana	21,7	21,2	21,7	-0,5
4 · Unas veces a la semana	25,9	35,4	18,0	17,4
5 · Una vez al día	24,6	34,7	18,4	16,3
6 · Varias veces al día	30,4	40,4	19,5	20,9
7 · Una vez por hora	32,4	46,1	21,4	24,7
8 · Más de una vez por hora	35,3	45,8	22,5	23,3

Nota. Muestral, sin ponderar. Segmentos de nivel 1 y 2 de `ArbolPrevalencia`. Fuente: F3/resultados/prevalencia_segmentos_f3.csv.